

What FHIR Is — and How to Implement It in PilotFish

Research note for the PilotFish Sandbox. Purpose: replace the directory-listener demo pattern with a real FHIR REST design before rebuilding the interface.

1. What FHIR actually is

FHIR (Fast Healthcare Interoperability Resources) is an HL7 standard for exchanging healthcare data as discrete **resources** (Patient, Observation, Encounter, Claim, etc.) over modern web APIs. The normative exchange model is **RESTful FHIR**: HTTP verbs against a service base URL, with JSON or XML payloads (`application/fhir+json` / `application/fhir+xml`).

FHIR is **not** “drop a file in a folder.” Directory pickup can be a Sandbox shortcut for non-FHIR demos, but it is not how FHIR systems talk to each other. Partners call HTTP endpoints; they expect HTTP status codes, Location headers, OperationOutcome errors, and CapabilityStatements.

2. How FHIR “messages” / transactions work

2.1 Primary model: RESTful interactions (usually synchronous)

Per HL7 FHIR R4 HTTP API, each resource type supports a common set of interactions. Default use is **synchronous request/response**: the client waits on the HTTP connection for the result. An async pattern also exists (Prefer: respond-async) for long-running work.

Level	Interaction	HTTP	Meaning
Instance	read / vread / update / patch / delete / history	GET / PUT / PATCH / DELETE	Operate on one resource by id
Type	create / search / history	POST / GET	Create or query a resource type
System	capabilities / batch / transaction / search	GET / POST	Server metadata or multi-resource commit

Canonical examples

```
POST [base]/Patient
→ create Patient; expect 201 Created + Location + resource (or Outcome).

GET [base]/Patient/{id}
→ read; expect 200 + Patient or 404 + OperationOutcome.

GET [base]/Patient?identifier=MRN|10001
→ search; expect Bundle type=searchset.

POST [base]/ with Bundle type=transaction
→ atomic multi-resource create/update/delete; response Bundle type=transaction-response.
```

2.2 Bundle batch vs transaction

- **batch** — each entry processed independently; partial success allowed; response type batch-response.
- **transaction** — all-or-nothing; server commits none if any entry fails; response type transaction-response.
- Entries carry `request.method` + `request.url` (and usually a resource body for POST/PUT).

2.3 FHIR Messaging (optional, different channel)

FHIR also defines **Messaging**: a Bundle with type=message whose first entry is a MessageHeader, exchanged over HTTP POST to a mailbox URL, queues, or other transports. That is closer to “event/message” semantics (admit, lab result notice). Most modern interoperability (Patient Access, Provider Access, Payer-to-Payer, prior auth APIs, CMS-0057-F style APIs) centers on **RESTful resource APIs**, not HL7v2-style directory drops. Messaging can be a later Sandbox route; REST CRUD/search/transaction should be the first real demo.

2.4 Is FHIR “real-time”?

Yes in the practical sense: clients invoke HTTP interactions and get immediate synchronous responses for create/read/search/transaction. That is interactive/request-driven, not a poll of a file share. Separately, systems may use subscriptions, async bulk (\$export), or messaging for deferred work — but the core API is live HTTP.

3. What was wrong with the directory-listener demo

- Partners never “drop XML envelopes” into a folder for FHIR; they call REST endpoints.
- No HTTP status codes, Location, ETag/versioning, or OperationOutcome contract.
- No CapabilityStatement describing supported resources/interactions.
- Validation wrapping metadata around RawFhir was a demo envelope — not on-the-wire FHIR.
- Useful as a temporary Sandbox scaffold; **not** a FHIR interface design to keep.

4. How PilotFish implements FHIR REST

PilotFish does **not** ship a dedicated “FHIR Listener” class. FHIR is a **format + REST/HTTP connectivity** problem: use the RESTful Web Service modules for the wire, JSON/XML transformers and the FHIR Format Builder for structure/mapping.

4.1 Modules (looked up in PilotFish_V2 / modules.conf)

Role	FQCN	Notes
Inbound REST	com.pilotfish.eip.modules.http.rest.RESTfulWebServiceListener	Path /eip/rest/{ServiceName}/{Resource}/{id}. Config: SERVICE_NAME, POST/GET/PUT/DELETE_SUPPORTED, Synchronous, Timeout, USER_NAME/PASSWORD, SupportedResources.
Sync reply	com.pilotfish.eip.modules.internal.SynchronousResponseTransport	Returns body to waiting REST/HTTP listener. Pair with Synchronous=true.
HTTP status	com.pilotfish.eip.modules.http.HttpResponseCodeProcessor	Sets com.pilotfish.HTTPResponseCode (201/200/404/400/...).
Response headers	com.pilotfish.eip.modules.http.AddHttp ResponseHeadersProcessor	e.g. Content-Type: application/fhir+json, Location.
JSON↔XML	com.pilotfish.eip.modules.json.JSONTransformationProcessor	Generic JSON tree ↔ PilotFish JSON XML — not FHIR profile validation by itself.
Outbound FHIR call	com.pilotfish.eip.modules.http.rest.RESTfulWebServiceTransport	RESOURCE_PATH, METHOD_TO_EXECUTE, ACCEPT_MEDIA_TYPE=application/fhir+json.
FHIR schemas (design-time)	FHIR Format Builder (xcs-format-fhir)	GUI mapping aid; versions in tree include v4.0.0 (R4). Not a runtime REST module.

Proven Sandbox sync-HTTP pattern (non-FHIR): CRL Plus American Income Life uses HttpPostListener + SynchronousResponseTransport. V2 unit-test routes exercise RESTfulWebServiceListener with Synchronous=true. No Clients demo has smoke-tested RESTfulWebServiceListener on pilotfish-eip:23R1 yet — that is the next build risk to retire.

4.2 URL shape on eiPlatform

```
/eip/rest/{ServiceName}/{ResourceName}/{ResourceID}
```

Example service name **fhir**:

```
POST /eip/rest/fhir/Patient
```

```
GET /eip/rest/fhir/Patient/pat-alice-001
```

Listener attributes include com.pilotfish.HttpMethodName, ResourceName, ResourceID, query parameters, and headers – route logic branches on those.

5. Recommended Sandbox target architecture

Rebuild the FHIR demo as a **small FHIR REST façade** (not a full EHR FHIR server).

- **Route A — FHIR REST Patient API (sync):** RESTfulWebServiceListener (SERVICE_NAME=fhir, Synchronous=true, GET+POST) → branch on method → store/retrieve Patient JSON (SQL or file store) → set status/headers → SynchronousResponseTransport with application/fhir+json.
- **Create:** POST body Patient → validate required elements → assign/persist id → 201 + Location + Patient.
- **Read:** GET by id → 200 Patient or 404 OperationOutcome.
- **Optional next:** search (GET Patient?identifier=... → Bundle searchset); transaction Bundle POST to base; outbound RESTfulWebServiceTransport to an external FHIR server.
- **Web UI:** act as FHIR client (curl-like) hitting LAN REST URLs — not a directory drop form.
- **LAN:** publish on 0.0.0.0 and set LAN_HINT to http://192.x.x.x:<port>/ per playbook §1.1.

Happy-path sequence (create)

1. Client POST application/fhir+json Patient to /eip/rest/fhir/Patient
2. Listener accepts, Synchronous wait starts
3. Route validates resourceType=Patient + identifier/name rules
4. Persist resource; set HTTPResponseCode=201; Location header
5. SynchronousResponseTransport returns Patient JSON to client
6. Optional side effect: BI SQL audit row (after successful persist – no claim-before-complete)

6. Validation & honesty

- Demo may use heuristic/XSLT or schema checks — label clearly as not full StructureDefinition/IG validation.
- Prefer returning OperationOutcome on failure with 4xx, not silent file kickout-only behavior.
- Do not claim CMS-0057-F / SMART-on-FHIR / OAuth compliance without implementing those gates.
- CapabilityStatement can start minimal (Patient create+read) and grow.

7. Proposed rebuild phases

Phase	Deliverable	Exit criteria
0 — Research (this PDF)	Agree REST-first design; retire directory pattern as the FHIR story.	Stakeholder OK to rebuild.
1 — REST smoke	RESTfulWebServiceListener + sync echo/Patient create on 23R1.	curl POST/GET from localhost + 192.x LAN works.
2 — Patient API	Persist store, 201/200/404, OperationOutcome, Web UI as client.	Happy + fail paths documented; route PDF regenerated.
3 — Expand	Searchset and/or transaction Bundle; optional outbound FHIR client route.	Samples + DESIGN.md risks updated.

8. Open questions before rebuild

- Confirm RESTfulWebServiceListener loads cleanly inside pilotfish-eip:23R1 WAR (first spike).
- Persistence: SQL JSON column vs filesystem FHIR store vs both (audit vs source of truth).
- Auth: demo basic auth only, or mock bearer token header check?
- Scope freeze for v1: Patient create+read only, or include Bundle transaction?
- Keep / deprecate current directory demo artifacts after REST rebuild?

9. References

- HL7 FHIR R4 HTTP API — <https://www.hl7.org/fhir/R4/http.html>
- HL7 FHIR R4 Bundle — <https://hl7.org/fhir/R4/bundle.html>
- PilotFish RESTful Web Service Listener — <https://healthcare.pilotfishtechnology.com/restful-listener-configuration/>
- PilotFish FHIR / CMS interoperability overview — <https://healthcare.pilotfishtechnology.com/fhir-integration-cms-0057-f-compliance/>
- In-repo module sources: PilotFish_V2 .../modules/http/rest/RESTfulWebService*.java
- Sandbox playbook: docs/INTERFACE_CONSTRUCTION_PLAYBOOK.md (§1.1 LAN, V1 runtime policy)

Bottom line: implement FHIR as a synchronous RESTful Web Service on PilotFish (listener → process → SynchronousResponse), not as a DirectoryListener. Rebuild the Sandbox demo against that contract after Phase 1 smoke proves the module on 23R1.